

Hochschule Esslingen

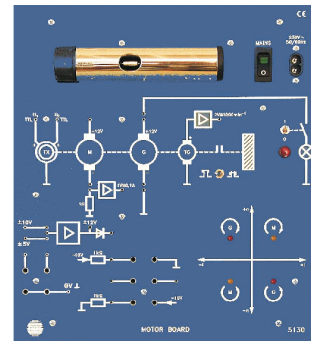
Fakultät Informationstechnik

Labor Regelungstechnik 2

Versuch: Motorboard

Bitte verwenden Sie

Matlab 2021a



Der Laborbericht ist spätestens eine Woche nach dem Labortermin abzugeben. Mangelhafte Vorbereitung beim Labortermin, verspätete oder unvollständige Abgabe des Laborberichts führen zum Nichtbestehen des Labors. Das Labor muss dann im folgenden Semester wiederholt werden.

Mit der Abgabe dieses Laborberichts und Ihrer Unterschrift bestätigen Sie, dass Sie den Bericht und die zugrundeliegenden Berechnungen, Programme und Simulationen selbstständig ohne Hilfe Dritter angefertigt haben.

Laborbericht von: _____

Laborpartner: _____

Motorboard Nr.: _____ Labortermin: _____

Die Laborteilnehmer sollten jeweils ihre **eigene handschriftlich** ausgearbeitete Laborvorbereitung in die Laborveranstaltung mitbringen.

Beachten Sie im Labor die aushängenden Sicherheitsvorschriften und die Sicherheitsunterweisung.

Inhaltsverzeichnis

1	Einleitung	3
2	Praktische Probleme bei der Drehzahl- und Lageregelung	4
2.1	Positions-messung, Rechen- und Abtastzeit	4
2.2	Messung der Sprungantworten der Regler und der Rechenzeiten	6
2.3	Haft–Gleitreibungseffekte bei der Positionsregelung	7
2.4	Genauigkeit der Positionsregelung	8
2.5	Positionsregelung mit PI–Regler?	9
3	Automatische C/C++–Codeerzeugung	11
3.1	Konfiguration des Codegenerators	11
3.2	Einbau und Test des automatisch generierten Codes auf dem Dragon12	14
4	Umstellung der Mikrocontroller–Implementierung auf Integer–Arithmetik	17
4.1	Probleme der Integer–Arithmetik	17
4.2	Entwurf der Regler in Integer–Arithmetik	18
4.2.1	Entwurf der Reglerskalierungen im Blockschaltbild	19
4.2.2	Rundungsprobleme, Überläufe und Genauigkeit beim Positionsregler	20
5	Anhang: Analyse von Messdaten	25
5.1	Software-Oszilloskop	25
5.2	Bedienung des Digital–Speicher–Oszilloskops	26
6	Anhang: Bedienoberfläche des HCS12–Programms	27
7	Anhang: Anschlüsse Motorboard - Mikrocontroller	28

Hinweis:

Die Simulink–Modelle für diesen Laborversuch und die hier abgebildeten Diagramme wurden mit Matlab R2021a erstellt. Neuere Versionen sind in der Regel aufwärtskompatibel, können sich aber in Details unterscheiden. Bei Problemen verwenden Sie bitte die Version R2021a.

1 Einleitung

Dies ist der zweite Teil des Versuchs Motorboard. Im ersten Teil wurde die Regelstrecke untersucht, deren Parameter bestimmt und eine Drehzahl- und Positionsregelung entworfen. In diesem Teil wird die Regelung weiter analysiert, Möglichkeiten der automatischen C-Codegenerierung untersucht und Implementierungsaspekte bei einfachen Mikrocontrollern betrachtet.

Das (vereinfachte) Blockschaltbild der in Kaskadenstruktur aufgebauten Positions- und Drehzahlregelung ist in Abb. 1 dargestellt. Die vollständige Beschreibung der Regelstrecke, des Simulationsmodells, des Dragon12-Programms sowie die Anschlussbelegung und Hinweise zur Labordurchführung und zum Laborbericht finden Sie im Umdruck zum ersten Teil des Labors.

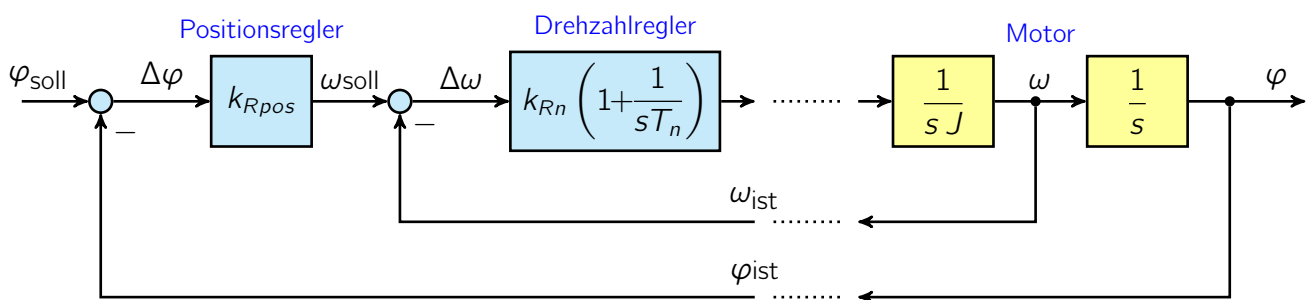


Abbildung 1: Positions- und Drehzahlregelkreis (stark vereinfacht)

Bitte verwenden Sie zur Implementierung der Regler unabhängig von Ihren Ergebnissen aus dem ersten Teil des Labors in der Datei *params.m* die folgenden Werte:

Drehzahlregler: $k_{Rn} = 1.2$ und $T_{Nn} = 51.1\text{ms}$

Positionsregler: $k_{Rpos} = 0.3$

Die Abtastzeit der Regler bleibt für Teil 2 auf $T = T_{RTI} = 1536\mu\text{sec}$ eingestellt.

Tragen Sie diese Werte bitte in die Dateien *params.m* und *main.c* ein!

Vorbereitungsaufgabe 1: Simulation der Drehzahl- und der Positionsregelung

Simulieren Sie mit diesen Reglerparametern im Simulink-Modells *Regelkreis_ZK.slx* die Sprungantworten ...

- ... der Drehzahlregelung bei einem Sprung der Solldrehzahl von 1000 auf 3000 U/min,
- ... der Positionsregelung bei einem Sprung der Sollposition von 0 auf 10 U.

Zoomen Sie so in die Sprungantworten hinein, dass der Einschwingvorgang bei der steigenden Flanke des Drehzahl- bzw. Positionswertes jeweils bildfüllend zu sehen ist (Zeitbereich ungefähr 0,9 ... 1,3 sec). Fügen Sie die beiden Bilder in Ihren Laborbericht ein.

2 Praktische Probleme bei der Drehzahl- und Lageregelung

2.1 Positionsmessung, Rechen- und Abtastzeit

Zur Positionserfassung verwenden wir den Positionsgeber TX auf dem Motorboard, der zwei um 90° versetzte Rechtecksignale U_1 und U_2 mit je 100 Pulsen je Umdrehung, insgesamt also 400 Signalfanken je Umdrehung, erzeugt (vergl. Abschnitt 3 im ersten Teil des Laborversuchs). Die Positionserfassung arbeitet folgendermassen:

- Die beiden Rechtecksignale werden mit zwei Capture-Eingängen der Timereinheit des Mikrocontrollers verbunden.
- Die Timereinheit wird so konfiguriert, dass bei jeder Signalfanke eine Interrupt-Service-Routine ISR aufgerufen wird.
- Die ISR erkennt aus der Kombination der beiden Rechtecksignale die Drehrichtung und zählt bei jeder Signalfanke entsprechend einen internen Zähler auf- bzw. abwärts. Der Zählerstand entspricht dann dem gewünschten Positionsmesswert.

Bei hohen Drehzahlen wird allerdings der zeitliche Abstand der Impulsflanken so klein, dass der Mikrocontroller die Interrupts nicht mehr schritthaltend verarbeiten kann. Die Verarbeitung funktioniert nur zuverlässig, wenn der Abstand deutlich größer ist als die Dauer der ISR, so dass auch noch ausreichend Zeit für die Analog-Digital-Umsetzung und die Berechnung der Regler bleibt.

Vorbereitungsaufgabe 2: Berechnung von Zeiten zur Positionsdekodierung

Berechnen Sie den Zeitabstand zwischen zwei Signalfanken des Positionsgebers bei einer Motordrehzahl $n_0 = 7800 \text{ U/min}$ (Leerlaufdrehzahl des Motors bei Nennspannung).

Dauer einer Motorumdrehung: $T_{rot} =$

Anzahl Signalfanken je Umdrehung: $N = 400 \text{ 1/Umdrehung}$

Zeitabstand der Signalfanken: $T_{interrupt} =$

Wenn man bedenkt, dass eine Multiplikation zweier Float-Variablen auf dem HCS12-Mikrocontroller, die z.B. notwendig wäre, um den Zählerstand in einen Winkelwert in Grad oder Radianten umzurechnen, etwa $12 \mu\text{sec}$ benötigt, ist klar, dass die Interrupt-Lösung bei hohen Drehzahlen nicht mehr sinnvoll eingesetzt werden kann.

Bei hohen Drehzahlen wird daher auf eine weitere Peripheriekomponente des HCS12-Mikrocontrollers umgeschaltet, den sogenannten Pulsakkumulator. Dieser zählt die Signalfanken in einem 8-bit Register, das zyklisch im Raster des RTI-Interrupts ausgelesen und

wieder zurückgesetzt wird. Allerdings gibt es auch hier eine Zeitgrenze. Damit die Positionserkennung zuverlässig funktioniert, muss die RTI-Zykluszeit so kurz sein, dass es nicht zu einem Überlauf dieses 8-bit Zählers kommt.

Berechnen Sie die maximal zulässige RTI-Zykluszeit so, dass bei der Leerlaufdrehzahl n_0 der Zähler gerade den maximalen Zählerstand von 255 erreicht:

Max RTI-Zykluszeit: $T_{RTI,max} =$

Von dieser theoretischen Grenze muss in der Praxis ein ausreichender Sicherheitsabstand eingehalten werden. In unserem Projekt arbeiten wir daher mit $T_{RTI} = 1536\mu\text{sec}$.

Da auch die Regelalgorithmen mit dieser kleinen Abtastzeit arbeiten, sollte sich das Verhalten einer Quasi-zeitkontinuierlichen Regelung ergeben. Bei der Auslegung des Positionsreglers und bei der Dimensionierung des Drehzahlreglers im ersten Teil des Laborversuchs haben wir jeweils eine gewünschte Phasenreserve von 80° bzw. 60° vorgegeben. Dass die Regler anschließend zeitdiskret implementiert werden, haben wir dabei nicht berücksichtigt. Dies ist zulässig, solange sich die Phasenreserve bei der Durchtrittskreisfrequenz ω_D , die sich bei der dortigen Auslegung ergeben hat, durch die Abtastung nicht wesentlich reduziert.

Vorbereitungsaufgabe 3: Überprüfung der Regler-Abtastzeiten

Berechnen Sie sowohl für den Positionsregler als auch für den Drehzahlregler die maximal zulässige Abtastzeit T_{max} , wenn sich die Phasenreserve bei der jeweiligen Durchtrittskreisfrequenz ω_D um maximal $\Delta\varphi_R = 5^\circ$ verringern darf:

Durchtrittskreisfrequenz Positionregler $\omega_{D,pos} =$

$\Rightarrow$ Maximale Abtastzeit für 5° Phasenverlust $T_{max} =$

Durchtrittskreisfrequenz Drehzahlregler $\omega_{D,rpm} =$

$\Rightarrow$ Maximale Abtastzeit für 5° Phasenverlust $T_{max} =$

Welcher Regler ist bezüglich der Abtastzeit kritischer?

Liegen wir mit unserer Abtastzeit von $T_{RTI} = 1536\mu\text{sec}$ auf der sicheren Seite?

2.2 Messung der Sprungantworten der Regler und der Rechenzeiten

Die Regelung kann nur dann sinnvoll funktionieren, wenn die Rechenzeit der Reglerfunktionen einschließlich der notwendigen Analog–Digital–Wandlungen (deutlich) kleiner ist als die gewählte Abtastzeit. Als nächstes sollen daher die einwandfreie Funktion der Regelkreise am Dragon12–Board überprüft und die Rechenzeiten gemessen werden.

Laboraufgabe 1: Überprüfen der Drehzahlregelung und Rechenzeit des Drehzahlreglers

Verwenden Sie Ihr eigenes Codewarrior–Projekt *Lab1_uC/Systemtechnik.mcp*, in dem Sie im ersten Teil des Laborversuchs bereits den PI–Drehzahlregler mit Anti–Windup–Massnahme eingebaut haben. Stellen Sie die Reglerparameter auf die im Kasten auf S.3 angegebenen Werte ein, laden Sie das Projekt auf das Dragon12–Board und testen Sie im Automatik–Betrieb der **Drehzahlregelung** Drehzahlsprünge zwischen 1000 und 3000 1/min. **Wenn die Dioden der Vierquadrantenanzeige wild blinken, schalten Sie das Board bitte möglichst schnell wieder aus** und überprüfen Sie Ihre Implementierung.

Messen Sie den Drehzahlverlauf mit Hilfe des Softwareoszilloskops (sh. S.25). Zoomen Sie so in das Bild hinein, dass Sie einmal die steigende und einmal die fallende Drehzahlflanke mit Ihren Simulationsergebnissen aus **Vorbereitungsaufgabe 1** vergleichen können. Falls die gemessenen Sprungantworten nicht ähnlich aussehen wie die Simulationsergebnisse, müssen Sie den Messaufbau und Ihr Programm überprüfen! Fügen Sie die beiden Bilder in Ihren Laborbericht ein.

Messung der Rechenzeit des Drehzahlreglers

Die Rechenzeit des Drehzahlreglers wird in der Betriebsart „Drehzahlregelung“ mit Hilfe eines Timer–Kanals des HCS12 erfasst und im LCD–Display in der zweiten Zeile angezeigt, wenn der automatische Stimulusgenerator aktiviert ist. Die gemessenen Werte enthalten die reine Rechenzeit des Reglers ohne die ADC–Wandlungen und den Stimulusgenerator. Messen Sie die Rechenzeiten für die Drehzahlregelung und tragen Sie den Werte in die linke Spalte in der Tabelle auf Seite 24 ein.

Als Positionsregler haben Sie im ersten Teil des Laborversuchs einen **P–Positionsregler** in Ihr Programm eingebaut. Der Positionsregler sollte sich in der Funktion *positionControl()* in *main.c* befinden. Die Reglerv Verstärkung soll hier und in allen folgenden Aufgaben wieder auf die im Kasten auf S. 3 angegebenen Werte eingestellt werden. Der Ausgang des Positionsreglers, d.h. die Soll Drehzahl für den Drehzahlregler, muss auf $\pm 7800 \text{ U/min}$ begrenzt sein (Normierungen im Mikrocontrollerprogramm berücksichtigen!).

Laboraufgabe 2: Messungen am Positionsregler und Vergleich mit der Simulation

Schalten Sie nun über die Bedientasten auf die **Positionsregelung** um und stellen Sie den Automatik–Betrieb ein, bei dem Sollpositionssprünge zwischen 0 und 10 U (Umdrehungen) gemacht werden. **Wenn die Dioden der Vierquadrantenanzeige wild blinken, schalten Sie das Board bitte möglichst schnell wieder aus** und überprüfen Sie Ihre Implementierung.

Wir wollen nun die Sprungantworten messen und die gemessene Position und die Drehzahl der Welle mit der Simulation vergleichen. Zur Messung wird wieder das Software-Oszilloskop verwendet. Im Mikrocontrollerprogramm wird am Ende der Interrupt-Service-Routine `myADC1Callback()` bei der Positionsregelung die Funktion

```
softScope((int16) pos_tick, (int16) (measured_rpm * CONV_ADC_RPM_UPMIN));
```

aufgerufen. Die Funktion überträgt bei jedem Abtastzeitpunkt die Ist-Position und die Ist-Drehzahl als 16bit-Integerwert in der programminternen Normierung an den PC. Die Werte können in Matlab mit dem Skript `SoftScope.m` angezeigt werden (sh. S.25).

Nehmen Sie die gemessenen Zeitverläufe in Ihren Laborbericht auf und vergleichen Sie die hier gemessenen Zeitverläufe für Position und für die Drehzahl mit den Simulationsverläufen aus der Vorbereitungsfrage 1. Falls die Verläufe stark voneinander abweichen, überprüfen Sie, ob Sie bei den Simulationen und bei den Messungen tatsächlich dieselben Reglerparameter und dieselben Sollwertsprünge eingestellt haben.

Messung der Rechenzeit

Messen Sie die Rechenzeit des Positionsreglers, die im Automatik-Betrieb der Positionsregelung im LCD-Display angezeigt wird, und tragen Sie den Wert in die linke Spalte in der Tabelle auf Seite 24 ein.

2.3 Haft-Gleitreibungseffekte bei der Positionsregelung

In Vorbereitungsaufgabe 1 wurde das Großsignalverhalten der Positionsregelung bei einem Sollwertsprung von 0 auf 10 U mit Hilfe des Simulink-Modells *Regelkreis_ZK.slx* simuliert. Dabei wird die Drehzahl so groß, dass praktisch nur die Gleitreibung eine Rolle spielt. Bei sehr kleinen Drehzahlen dagegen macht sich auch die Haftreibung bemerkbar.

Vorbereitungsaufgabe 4: Kleinsignalverhalten der Positionsregelung

Simulieren Sie einen Sollwertsprung von 0 auf $1/400 = 0,0025U$ zur Zeit $t_w = 1 \text{ sec}$. Die Ergebnisse werden von denen der vorigen Aufgabe deutlich abweichen. Es entsteht ein Verhalten, das so nicht zu erwarten war. Fügen Sie den Zeitverlauf der Position in Ihren Bericht ein.

Der Übergang zwischen Haft- und Gleitreibung im Bereich um die Drehzahl 0 ist ein stark nichtlinearer Effekt. Dieser **Stick-Slip-Effekt** kann dazu führen, dass sich statt einer konstanten Position eine Grenzschrägung einstellt. Verändern Sie in *params.m* denjenigen Parameter, der diese Haftreibung beschreibt so, dass dieser Effekt verschwindet. Führen Sie damit erneut eine Simulation aus und fügen Sie auch diesen Zeitverlauf der Position in Ihren Bericht ein. **Geben Sie den Namen des Parameters sowie seinen Wert** an, den Sie bei Ihrer Simulation eingestellt haben:

Stellen Sie sicher, dass der Parameter in *params.m* für die folgenden Aufgaben wieder den ursprünglichen Wert besitzt.

2.4 Genauigkeit der Positionsregelung

Der Positionsregler wurde im ersten Teil des Laborversuchs als P–Regler nach dem Verfahren mit Phasenreserve entworfen. Wegen des fehlenden I–Anteils im Regler soll hier die Genauigkeit der Positionsregelung analysiert werden.

Vorbereitungsaufgabe 5: Stationäre Genauigkeit des Positionsreglers

Weshalb genügt unter **idealen Verhältnissen, d.h. bei idealer Positions- und Drehzahlmessung**, ein reiner P–Regler als Positionsregler, um im eingeschwungenen Zustand eine verschwindende Regelabweichung $\Delta\varphi(t \rightarrow \infty) = 0$ sicherzustellen? Betrachten Sie zur Beantwortung dieser Frage das (vereinfachte) Blockschaltbild des Regelkreises in Abb. 1 und geben Sie die stationären Werte der unten angegebenen Signale an.

$$\omega(\infty) = \quad \omega_{ist}(\infty) = \quad \Delta\omega(\infty) =$$

$$\omega_{soll}(\infty) = \quad \Delta\varphi(\infty) =$$

Im ersten Teil des Laborversuchs in Laboraufgabe 2 haben wir festgestellt, dass die Drehzahlmessung einen Messfehler verursacht, d.h.

$$\omega_{ist} = \omega_{ofs} + k_n \cdot \omega = 2\pi(n_{ofs} + k_n \cdot n).$$

Dabei entspricht $\omega_{ofs} = 2\pi n_{ofs} \neq 0$ einem Offsetfehler, $k_n \neq 1$ einem Steigungsfehler.

Führt der Messfehler des Tachogenerators (Drehzahlmessung) zu einer bleibenden Abweichung der Positionsregelung? Geben Sie eine Formel zur Berechnung der bleibenden Regelabweichung $\Delta\varphi(t \rightarrow \infty)$ der Positionsregelung an. Die Positionsmessung selbst darf weiterhin als ideal angenommen werden, d.h. $\varphi_{ist} = \varphi$.

$$\omega(\infty) = \quad \omega_{ist}(\infty) = \quad \Delta\omega(\infty) =$$

$$\omega_{soll}(\infty) = \quad \Delta\varphi(\infty) =$$

Um die Lageregelung genauer zu machen, müsste der Offsetfehler der Drehzahlmessung kompensiert werden. Das wäre z.B. möglich, indem man den Offsetfehler misst und dann im Mikrocontrollerprogramm vom gemessenen Wert abzieht. Wenn der Offsetfehler zeitlich konstant ist, ist dies die einfachste Möglichkeit.

2.5 Positionsregelung mit PI-Regler?

Alternativ könnte man überlegen, den P-Positionsregler durch einen PI-Regler zu ersetzen. Wie dies die Stabilität und das dynamische Verhalten des Regelkreises beeinflussen würde, soll mit Hilfe der Wurzelortskurve untersucht werden. Der offene Positionsregelkreis (Abb. 1) enthält folgende dominierenden Pole bzw. Nullstellen, die alle reell sind (Abb. 2):

- Einen Pol bei $s_{\infty,1} = 0$ durch den Integrator zwischen Drehkreisfrequenz $\omega = 2\pi n$ und Position φ .
- Einen Pol, der von der mechanischen Zeitkonstante des Motors im Drehzahlregelkreis resultiert, bei $s_{\infty,2} \approx -18.5 \frac{\text{rad}}{\text{sec}}$
- Einen weiteren Pol bei $s_{\infty,3} \approx -120 \frac{\text{rad}}{\text{sec}}$, der ebenfalls von der Drehzahlregelung stammt.
- Eine Nullstelle bei $s_0 \approx \frac{1}{T_{n,n}} \approx -20 \frac{\text{rad}}{\text{sec}}$ durch den PI-Drehzahlregler. Diese Nullstelle sollte eigentlich die mechanische Zeitkonstante kompensieren, ist dafür aber bei den in diesem Teil des Labors gewählten Reglerparametern etwas zu schnell.

Die vergleichsweise kleinen Zeitkonstanten des PWM-Filters, der Drehzahlmessung sowie des elektrischen Teils des Motors (alle $\leq 2\text{msec}$), wurden dabei vernachlässigt.

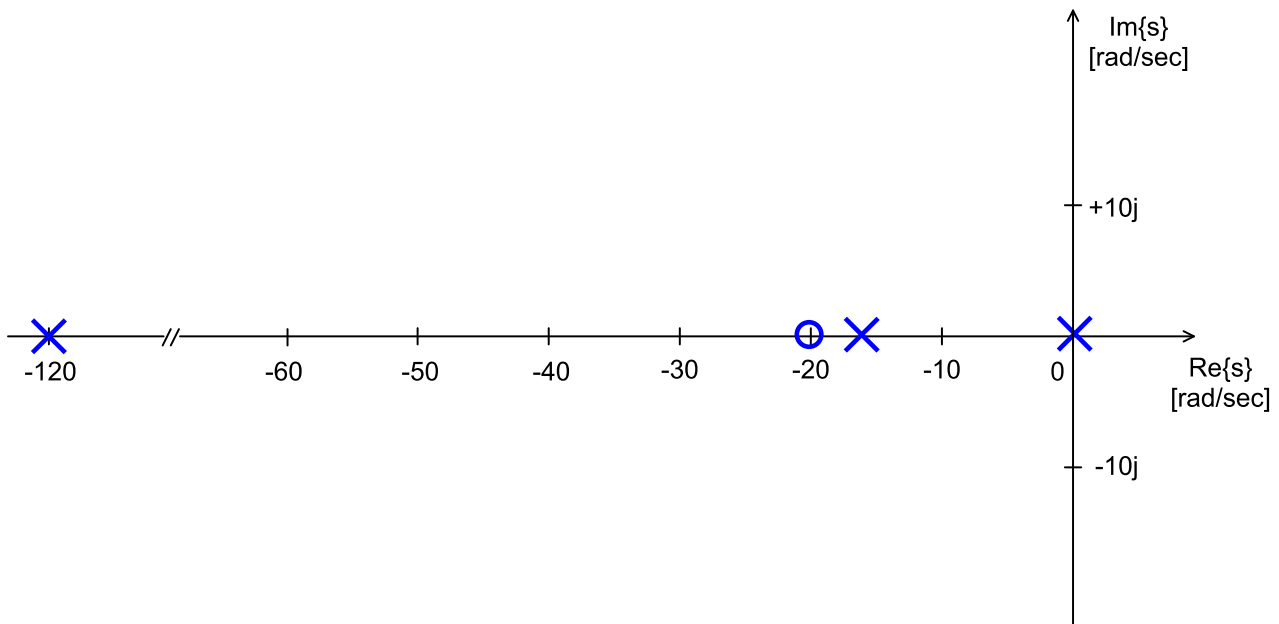


Abbildung 2: (Vereinfachte) Wurzelortskurve des Positionsregelkreises mit P-Regler

Vorbereitungsaufgabe 6: Wurzelortskurve des Positionsregelkreises

In Abb. 2 soll die Wurzelortskurve (WOK) des Positionsregelkreises mit P-Positionsregler skizziert werden. Die Pole und Nullstellen sind bereits eingezeichnet.

- Wieviele Äste wird die WOK haben und wieviele davon gehen ins Unendliche?
- Geben Sie den Schnittpunkt σ_a und die Winkel φ_a der Asymptoten für diejenigen Äste der WOK an, die ins Unendliche gehen und zeichnen Sie diese Asymptoten ein.
- Markieren Sie in Abbildung 2 diejenigen Abschnitte auf der reellen Achse, auf denen Äste der WOK liegen.
Deuten Sie durch Pfeile an, in welcher Richtung die WOK in der Umgebung der Pole bzw. Nullstelle verläuft, wenn die Reglerverstärkung $k_{R,pos} \uparrow$ des Positionsreglers vergrößert wird.
- In welchem Bereich auf der reellen Achse müssen Verzweigungs- bzw. Vereinigungspunkte der WOK liegen?
- Skizzieren Sie die vollständige WOK.

Wenn man statt des P-Positionsreglers einen PI-Regler

$$G_{R,pos} = k_{R,pos} \frac{1 + sT_{n,pos}}{sT_{n,pos}}$$

einsetzt, ergibt sich ein weiterer Pol bei $s_{\infty,PI} = 0$ und eine Nullstelle bei $s_{0,PI} = -\frac{1}{T_{n,pos}}$.

Während der zusätzliche Pol fix ist, kann die neue Nullstelle durch die Wahl von $\frac{1}{T_{n,pos}}$ theoretisch auf der negativen reellen Achse frei platziert werden, etwa a) zwischen die beiden langsamen Pole bei $s_{0,PI} = -12.5 \frac{\text{rad}}{\text{sec}}$ oder b) zwischen die beiden schnellen Pole z.B. bei $s_{0,PI} = -25 \frac{\text{rad}}{\text{sec}}$. Verwenden Sie das Matlab-Skript *wok_Positionsregelkreis.m*, um die WOK für die Dimensionierungen a) bzw. b) zu zeichnen. Fügen Sie beide Diagramme in Ihren Laborbericht ein. Sehen Sie eine Chance, mit dem PI-Positionsregler einen stabilen Regelkreis aufzubauen (Begründung)? Wird die Sprungantwort überschwingen?

3 Automatische C/C++-Codeerzeugung

Die modellbasierte Vorgehensweise mit Werkzeugen wie Matlab/Simulink erlaubt eine schnelle Systementwicklung mit hohem Abstraktionsgrad. Die anschließende manuelle Umsetzung in einen echtzeitfähigen Embedded-C/C++-Programmcode kann bei komplexen Aufgabenstellungen allerdings immer noch langsam, unflexibel und fehlerträchtig sein. Daher wird zunehmend versucht, diesen Schritt durch automatisierte Codegenerierung zu ersetzen.

3.1 Konfiguration des Codegenerators

Um diese Vorgehensweise kennenzulernen, verwenden wir das Simulink-Projekt *Regelkreis_ZD.slx*. Dieses Projekt enthält wiederum die Positions- und Drehzahlregelung für das Motorboard, wobei in beiden Reglern nun zeitdiskrete Simulinkblöcke eingesetzt werden und beide Regler jeweils in ein sogenanntes Subsystem *posRegler* bzw. *nRegler* verschoben wurden. Das „Innenleben“ eines Subsystems können Sie sich durch Doppelklick ansehen. **Bevor Sie** das Modell simulieren oder daraus **C/C++-Programmcode erzeugen**, müssen Sie wieder das **Matlab-Skript *params.m* ausführen**, das die Parameter für das Modell einstellt.

Wir wollen nun aus dem Modell den C/C++-Programmcode für die beiden Regler automatisch generieren. Der Simulink-Codegenerator lässt sich in weiten Grenzen konfigurieren. **Hinweis:** Diese Beschreibung und die Screenshots wurden mit **Matlab R2021a** erstellt, neuere Versionen können davon abweichen!

Öffnen Sie das Fenster *Model Settings* über **CTRL+E**.

- Unter *Hardware Implementation* (Abb. 3) muss der Mikrokontrollertyp ausgewählt und die Größe der Standarddatentypen eingestellt werden.

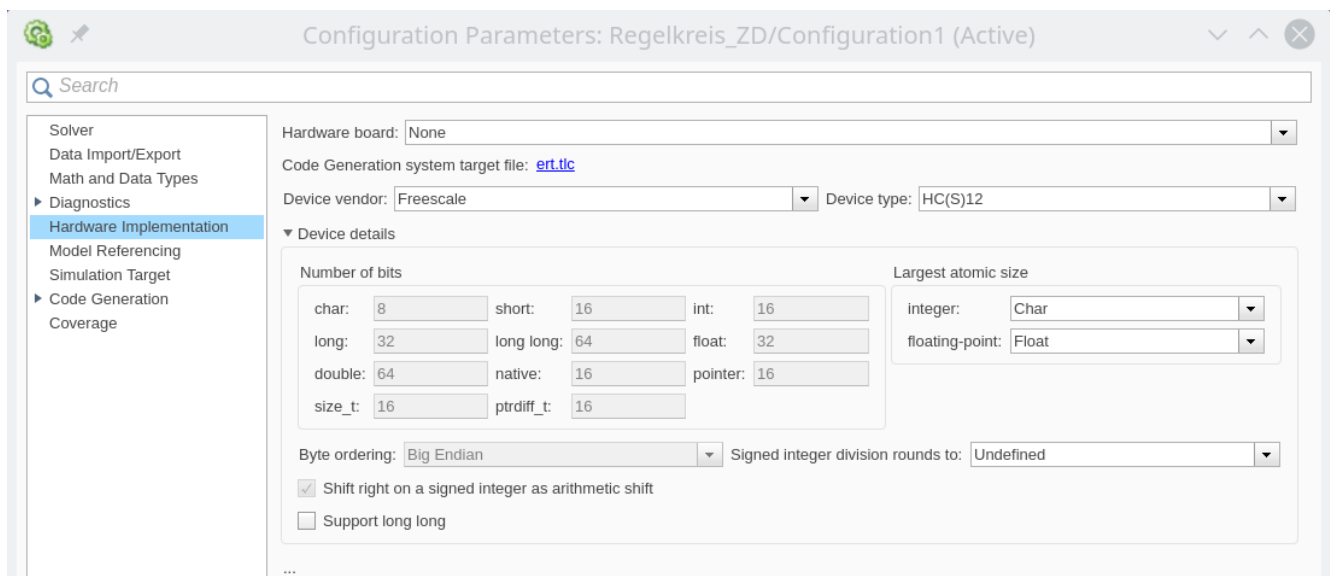


Abbildung 3: C/C++-Codeerzeugung Optionen 1

- Unter *Code Generation* (Abb. 4) wird die Programmiersprache C sowie die Zielumgebung *Embedded Realtime Target (ERT.tlc)* ausgewählt.

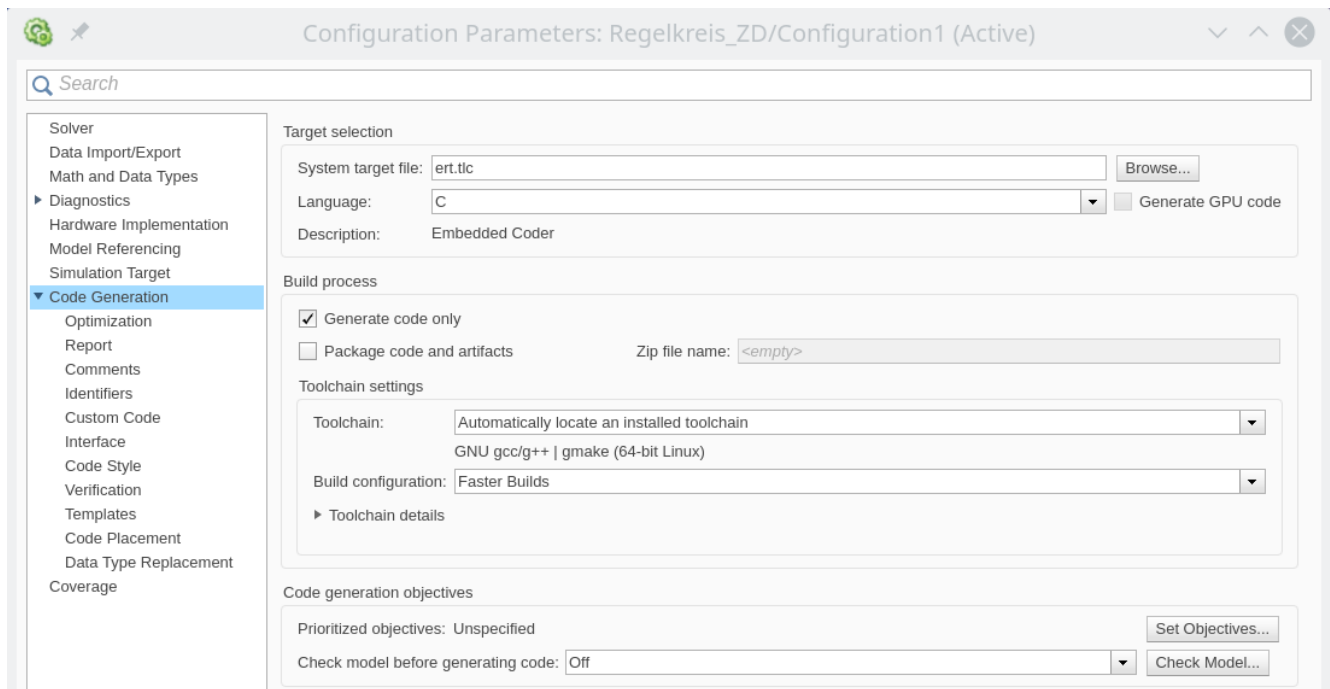


Abbildung 4: C/C++-Codeerzeugung Optionen 2

- Unter *Interface* (Abb. 5) wird ausgewählt, dass C89/90-ANSI-kompatibler Code unter Verwendung von *Floating-Point*-Variablen gewünscht wird, der im Wesentlichen aus einer Initialisierungsfunktion `<Name des Subsystems>_initialize()` sowie einer Funktion `<Name des Subsystems>_step()` besteht, die bei jedem Abtastschritt aufgerufen werden muss (*Single output/update function*).
- *Code Placement* (Abb. 6) gibt vor, dass neben der Code-Datei eine Header-Datei erzeugt wird, die die Deklaration der Datenstrukturen enthält (*Data definition* und *Data declaration*).

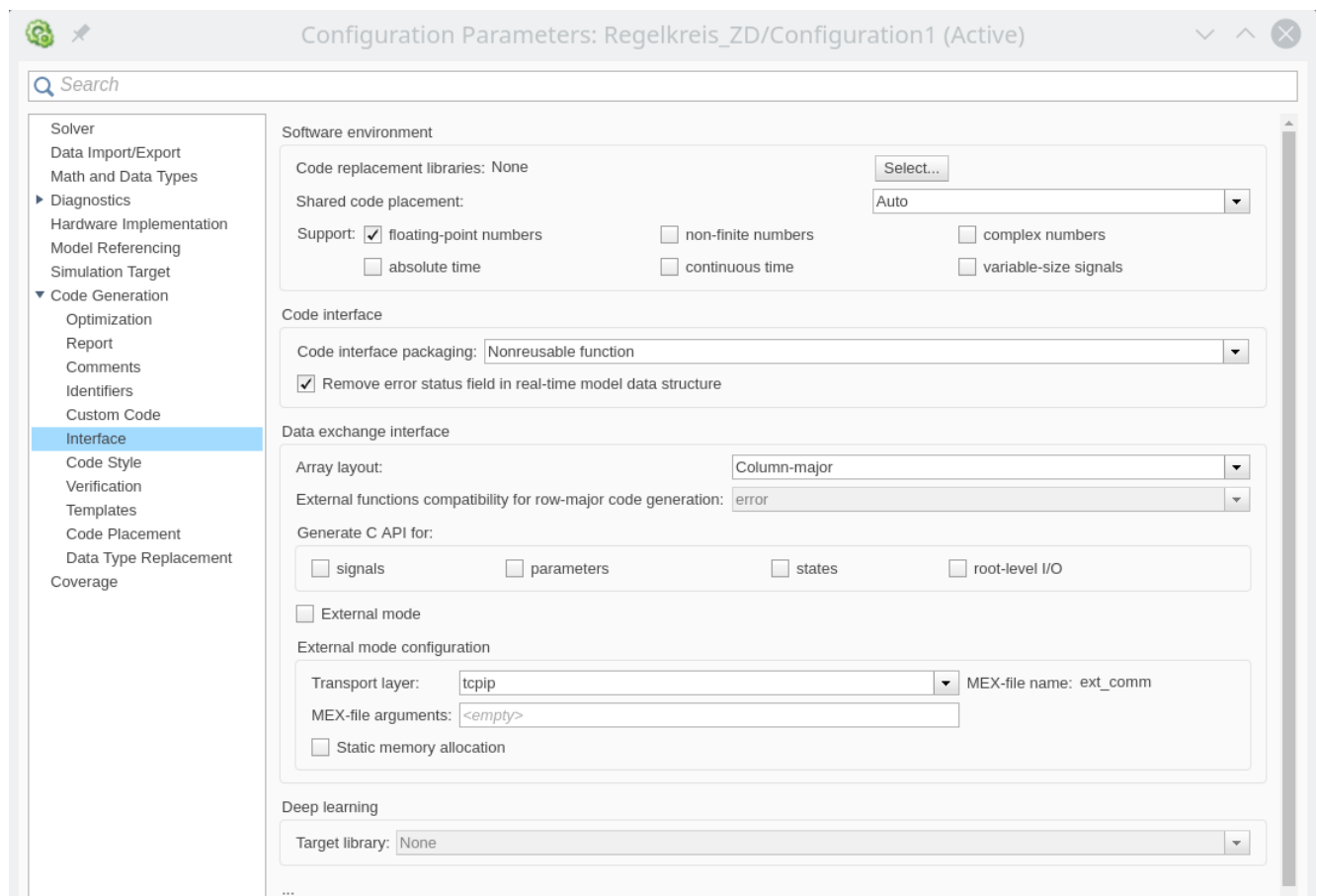


Abbildung 5: C/C++-Codeerzeugung Optionen 3

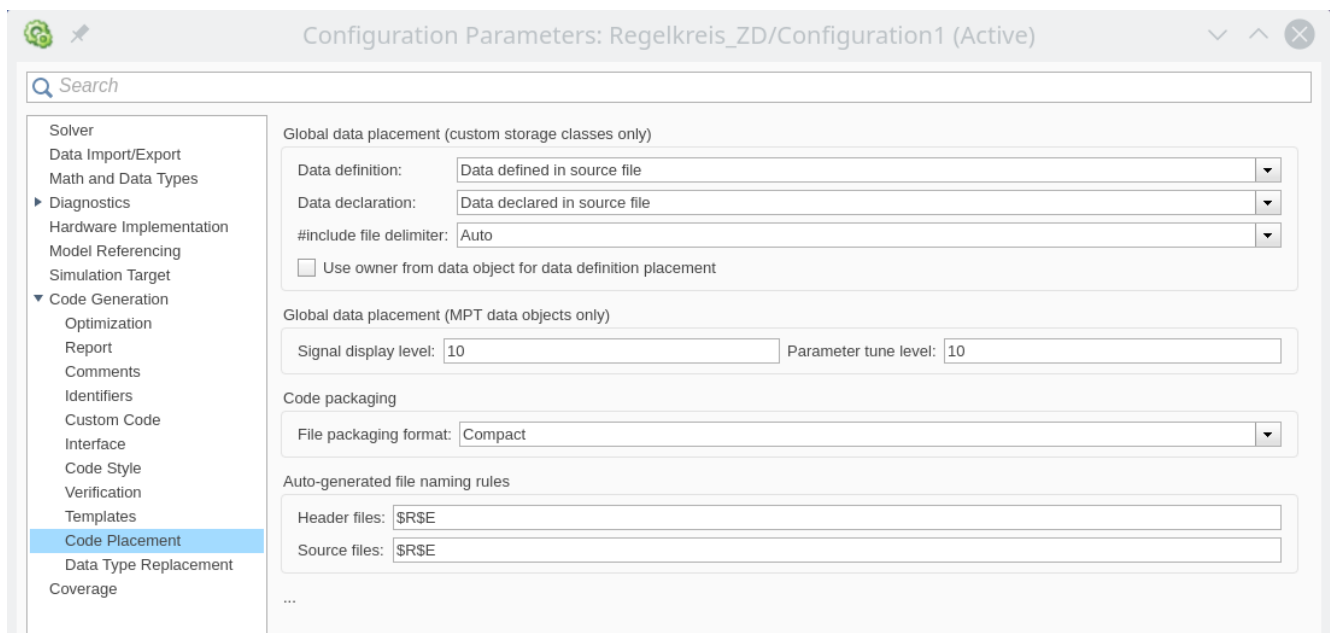


Abbildung 6: C/C++-Codeerzeugung Optionen 4

- Nachdem die Konfiguration abgeschlossen wurde, wird im Simulink-Modell jeweils mit der Maus rechts auf eines der beiden Subsysteme *posRegler* bzw. *nRegler* geklickt und in der Dialogbox *C/C++-Code - Build this Subsystem* ausgewählt. Der erzeugte Programmcode sowie einige Hilfsdateien werden in einem Unterverzeichnis <Name des Subsystems>_ert_rtw abgelegt. Ausserdem erscheint ein Fenster „Code Generation Report“, in dem man sich unter „Code Interface Report“ die Aufrufchnittstelle der erzeugten C-Funktionen und unter „Generated Code - Model Files“ den Programmcode ansehen kann.

3.2 Einbau und Test des automatisch generierten Codes auf dem Dragon12

Im Folgenden soll nun aus dem Simulink-Modell Programmcode generiert und in das Codewarrior-Projekt integriert werden.

Laboraufgabe 3: Codegenerierung für den Positions- und den Drehzahlregler

- Öffnen Sie das Simulink-Modell *Regelkreis_ZD.slx*. Setzen Sie die Parameter mit Hilfe des M-Skripts *params.m*. Simulieren Sie zunächst das Verhalten des Drehzahlreglers bei einem Sprung des Drehzahlsollwerts und anschließend des Positionsreglers bei einem Sprung des Positionsreglers und prüfen Sie, ob sich die Regelkreise wie erwartet verhalten.
- Prüfen Sie, ob der C/C++-Codegenerator wie oben beschrieben korrekt konfiguriert ist. Erzeugen Sie dann den C-Code für den Positionsregler (Simulink-Block *posRegler*) und für den Drehzahlregler (Simulink-Block *nRegler*).

- Öffnen Sie die Dateien *nRegler.c* und *nRegler.h* und versuchen Sie den Code, insbesondere die Funktionen *nRegler_step()* und die erzeugten Datenstrukturen grob zu verstehen.
- Der automatisch erzeugte Code für den Positionsregler befindet sich in den Dateien *posRegler.h* bzw. *posRegler.c*.

Im nächsten Schritt muss der automatisch erzeugte Programmcode in das Codewarrior-Projekt *uc.autocode/Systemtechnik.mcp* für das Dragon12-Board integriert werden. Der Stand dieses Projektes entspricht dem Codewarrior-Projekt aus dem ersten Laborversuch, bevor Sie dort begonnen haben, die Regler selbst in C zu programmieren.

- Öffnen Sie das Projekt und Rechtsklicken Sie in der Codewarrior-Verzeichnisansicht auf das *Sources*-Verzeichnis, wählen Sie *Add files*, navigieren Sie in das Verzeichnis Ihres Simulink-Projektes und fügen Sie die Dateien *../nRegler_ert_rtw/nRegler.c* und *nRegler.h* sowie *../posRegler_ert_rtw/posRegler.c* und *posRegler.h* in Ihr Codewarrior-Projekt ein.
- Öffnen Sie die Datei *main.c* im Editor und führen Sie folgende Änderungen durch:
 - Am Anfang der Datei hinter *#include „main.h“* müssen Sie die beiden Include-Dateien *posRegler.h* und *nRegler.h* einfügen.
 - In der Funktion *main()* vor der Endlosschleife *for(;;)...* müssen Sie die Aufrufe *posRegler_initialize()* und *nRegler_initialize()* zur Initialisierung der beiden Regler einfügen.

- Gehen Sie nun in die Funktion *positionControl()* und fügen Sie dort folgende Codezeilen ein:

```
posRegler_U.Soll = set_position; // Input signals
posRegler_U.Ist  = pos_tick;
posRegler_step();           // Control algo
set_rpm = posRegler_Y.StellB; // Output signal
```

Die ersten beiden Zeilen kopieren den Soll- und den Istwert in die Eingangsdatenstruktur *posRegler_U*, in der der automatisch erzeugte Code die Eingangssignale erwartet. Die dritte Zeile ruft die Funktion auf, in der der Regler in jedem Abtastschritt berechnet wird. Die vierte Zeile kopiert die Stellgröße aus der Ausgangsdatenstruktur *posRegler_Y* zurück in unsere globale Variable. Wenn Sie die Namen der Datenstrukturen mit dem Simulink-Modell vergleichen, sehen Sie, dass im generierten C-Code die Namen der Blöcke und Signale aus dem Simulink-Modell wieder auftauchen.

- Analog in der Funktion *rpmControl()*:

```
nRegler_U.Soll = set_rpm;           // Input signals
nRegler_U.Ist  = measured_rpm;
nRegler_step();           // Control algo
rpm_control_u = nRegler_Y.StellB; // Output signal
```

- Das Projekt sollte sich jetzt fehlerfrei übersetzen und auf das Dragon12-Board laden lassen. Testen Sie die Drehzahlregelung und die Positionsregelung.

Messen Sie wie in Laboraufgabe 1 die Sprungantwort des Drehzahlreglers und analog zu Laboraufgabe 2 die Sprungantwort des Positionsreglers. Beide Diagramme sollen Sie in Ihren Laborbericht aufnehmen. Falls die Sprungantworten deutlich von den Messungen in Laboraufgabe 1 und 2 abweichen, müssen Sie die Ursache suchen und beseitigen.

Messung der Rechenzeit

Messen Sie für den automatisch generierten Programmcode wiederum die Rechenzeiten in den Betriebsarten „Positionsregelung“ bzw. „Drehzahlregelung“, die im LCD-Display in der zweiten Zeile angezeigt wird, wenn der automatische Stimulusgenerator aktiviert ist. Tragen Sie diese Werte in die mittlere Spalte in der Tabelle auf Seite 24 ein. Ist der automatisch generierte Programmcode schneller oder langsamer als der Programmcode, den Sie selbst geschrieben haben?

4 Umstellung der Mikrocontroller–Implementierung auf Integer–Arithmetik

Vor allem kleinere Mikrocontroller wie der HCS12 haben keine Gleitkomma–ALU sondern verwenden eine Softwarebibliothek, die die Gleitkommaarithmetik in Festkommaalgorithmen für die Mantissen und Exponenten zerlegt. Derartige Berechnungen können im Vergleich zu Integer–Berechnungen extrem langsam sein. Für das Dragon12–Board mit einer CPU–Taktfrequenz von 24MHz können z.B. folgende Werte gemessen werden:

Operation	Datentyp: short int (16bit)	long int (32bit)	float (32bit)	double (64bit)
$c = a \pm b$	0,4 μ sec	0,9 μ sec	8,5 μ sec	21 μ sec
$c = a * b$	0,5 μ sec	3,5 μ sec	12 μ sec	175 μ sec
$c = a / b$	0,9 μ sec	6,0 μ sec	44 μ sec	225 μ sec

Ein PI–Regler inklusive Regeldifferenzbildung ohne Anti–Windup–Massnahme erfordert mindestens 2 Multiplikationen und drei Additionen/Subtraktionen:

```
e  = xSoll - xIst;           // Regeldifferenz
uI = uI + kI * e;           // I--Anteil
uP = kP * e;                // P--Anteil
u  = uP + uI;               // Stellgroesse
```

Bei unserer Abtastzeit von 1536 μ sec sollten wir uns definitiv keine Operation mit double–Variablen, aber auch nicht allzu viele float–Berechnungen erlauben. Für unsere Implementierung der P–PI–Regler–Kaskade, bei der alle Signale und Reglerparameter als float–Variable umgesetzt wurden, haben wir bereits knapp unter 300 μ sec Rechenzeit gemessen (ohne A/D–Wandlungen, Überstromschutz usw.). Die Autocode–Variante dauert noch länger.

4.1 Probleme der Integer–Arithmetik

Wenn aus Kostengründen keine schnellere CPU eingesetzt werden kann, muss der Algorithmus so verändert werden, dass die Berechnungen mit schnelleren Integer–Operationen erfolgen können. Dabei tauchen allerdings zwei mögliche Probleme auf:

- **Überläufe:** Während bei Additionen $a + b$ Überläufe dadurch vermieden werden, dass man die Variablen geeignet skaliert (z.B. für `int` $a, b, c \rightarrow |a, b| < 16768 \rightarrow |c = a + b| < 32768$) lassen sich Überläufe bei Multiplikationen praktisch nur vermeiden, indem man für das Ergebnis die doppelte Breite der Faktoren vorsieht, also z.B. für `int` a, b ; `long` $c = (\text{long}) a * (\text{long}) b$.

Bei längeren Berechnungen können Überläufe bereits bei den Zwischenergebnissen auftreten. Prüft man lediglich das Endergebnis auf Überlauf, lassen sich Überläufe in den Zwischenergebnissen nicht immer erkennen. Das Endergebnis ist dann nur scheinbar korrekt.

- **Rundungsprobleme:** Bei Reglern sind die Zahlenwerte häufig sehr klein. Der Verstärkungsfaktor des I-Anteils ist fast immer $k_I = k_P * T/T_n \ll 1$, da die Abtastzeit $T \ll T_n$ gewählt werden muss und nur selten $k_P \gg 1$ sein dürfte. Bei Integer-Arithmetik entspricht eine Multiplikation mit $k_I < 1$ jedoch einer Multiplikation mit 0, d.h. der I-Anteil ist praktisch nicht vorhanden.

Rundungsprobleme treten auch beim P-Anteil auf. Falls sich bei der Reglerauslegung ein Wert von z.B. $k_P = 1.5$ ergeben hat, hat man bei direkter Integer-Umsetzung ohne Umskalierung lediglich die Wahl, mit $k_P = 1$ oder $k_P = 2$ zu arbeiten. Die Abweichung vom theoretisch berechneten Wert und damit die Auswirkung auf das Regelverhalten ist erheblich.

Rundungsprobleme treten bereits bei der Berechnung der Regeldifferenz e auf. Sobald die Differenz bei Annäherung an den eingeschwungenen Zustand $e < 1$ wird, rundet die Integer-Arithmetik die Differenz auf 0 ab, was sowohl den P- als auch den I-Anteil bei diesen kleinen Werten faktisch abschaltet. Das „Abschalten“ führt dann dazu, dass die Regeldifferenz wieder über die Rundungsschwelle hinaus ansteigt usw. und der Regler wieder wirksam wird, wodurch die Regeldifferenz wieder unter die Rundungsschwelle sinkt. Der Regelkreis geht dann in eine Grenzschwingung geringer Amplitude, aber hoher Frequenz über.

Versucht man, die Berechnungen so umzuskalieren, dass alle Werte $\gg 1$ werden, um die Rundungsfehler zu reduzieren, riskiert man, dass Überläufe entstehen (siehe voriger Punkt).

4.2 Entwurf der Regler in Integer-Arithmetik

Um zu einer Integer-Realisierung zu kommen, betrachtet man alle Signale und Parameter des Reglers und skaliert diese so, dass zum einen die Genauigkeit ausreichend ist und zum anderen Zahlenbereichsüberläufe so weit wie möglich vermieden werden. Falls die Vermeidung von Überläufen nicht garantiert werden kann, verwendet man Sättigungsalgorithmen, die im Falle eines Überlaufes in die jeweilige Begrenzung laufen statt einen Wrap-Around zu produzieren. Ein solcher Zahlenbereichsüberlauf ist insbesondere in der Regelungstechnik kritisch, da sich dadurch der Regelsinn verkehrt und der Regelkreis instabil werden kann. In der Analogtechnik findet ein solcher Wrap-Around prinzipiell nicht statt, weil ein übersteuerter Verstärker in der jeweiligen Begrenzung verharrt.

Ein Wrap-Around tritt z.B. dann auf, wenn zu einer 4-bit Zahl $+7 = (0111)_2$ die Zahl 2 addiert wird. Hier ergibt sich die Zahl $9 = (1001)_2$. Falls nun aber keine Dualcodierung sondern eine Zweierkomplement-Codierung zugrundeliegt, ergibt sich hier als Ergebnis $(1001)_2 = -7$. Statt einer sehr großen positiven Stellgröße wird tatsächlich eine sehr große negative Stellgröße ausgegeben. Dies kann zu katastrophalem Fehlverhalten des Systems führen.

Zahlenbereichsüberlauf: Die Möglichkeit eines Zahlenbereichsüberlaufes muss bei allen Rechenoperationen geprüft werden. Hierfür eignet sich vor allem die Assembler-Programmierung. In Hochsprachen kann man einen Überlauf bei Addition/Subtraktion relativ einfach anhand der Vorzeichen der Operanden und des Ergebnisses erkennen. Z.B. tritt ein Überlauf auf falls $(+) + (+) = (-)$. In einem solchen Fall kann das „richtige“ Ergebnis nicht im vorgegebenen Zahlenbereich dargestellt werden. Man kann dann entweder die Bitbreite der entsprechenden Variablen erhöhen oder, falls dies nicht möglich oder gewünscht ist, als Ergebnis die größte positive Zahl verwenden, die

in der zugrundeliegenden Zahlencodierung noch dargestellt werden kann. Eine solches Vorgehen bezeichnet man als Sättigung.

Laboraufgabe 4: Sättigungsalgorithmus bei Addition

Ein möglicher Sättigungsalgorithmus für die Addition zweier `int` Zahlen ist:

```
int sat_add_int( int a, int b)
{
    int s = a + b;
    if( a > 0 && b > 0 && s < 0){
        s = INT_MAX;
    }else if( a < 0 && b < 0 && ( s > 0 || s == INT_MIN) ){
        s = -INT_MAX;
    }
    return(s);
}
```

Dabei fällt auf, dass die Zahlenbereichsgrenzen nicht hart codiert sind, sondern Symbole verwendet werden. Wo sind diese in C definiert? Weshalb schließt man offensichtlich die Zahl `INT_MIN` vom darstellbaren Zahlenbereich aus? Überlegen Sie sich insbesondere, was bei der Komplementbildung von `INT_MIN` entsteht.

Definition in:

`INT_MIN` ausgeschlossen weil:

Bei **Multiplikation** ist das Erkennen von Überläufen schwieriger. Daher schließen wir solche Überläufe algorithmisch aus, indem der Zahlenwert des variablen Operanden vor der Multiplikation mit der bekannten Reglerkonstanten durch Sättigungselemente begrenzt wird. Die Begrenzung ist so zu wählen, dass garantiert werden kann, dass das Ergebnis der Multiplikation in der gewählten Zahlencodierung darstellbar ist.

4.2.1 Entwurf der Reglerskalierungen im Blockschaltbild

Um zu einer Integer-Realisierung zu kommen, wird das Blockschaltbild des Reglers umgeformt, indem wir an passenden Stellen die Identität $2^{\nu} \cdot 2^{-\nu}$ einfügen. Weil das Vertauschen von Blöcken und die weiteren Blockschaltbildumformungen, die Sie in der Vorlesung kennengelernt haben, bei linearen Systemen Äquivalenzumformungen darstellen, können wir die beiden Blöcke dann durch das Blockschaltbild verschieben (Kommutativgesetz) und so beispielsweise andere Blöcke des originalen Blockschaltbildes zwischen sie bekommen. Durch Anwendung des Assoziativgesetzes können wir die Blöcke auch zu neuen Blöcken zusammenfassen. Diese Umformungen sind sinnvoll, weil das System durch die nachträgliche Veränderung der Zahlendarstellung nichtlinear wird und sich somit erhebliche Änderungen im Verhalten ergeben. Dies sei am folgenden Beispiel erläutert

(mit $e = 10$ und $k = 0.5$):

$$\begin{aligned}
 e \cdot k &= u \\
 \text{in float} &\Rightarrow 10 \cdot 0.5 = 5 \\
 \text{in integer} &\Rightarrow 10 \cdot 0.5 = 10 \cdot 0 = 0 \\
 \text{jetzt mit Skalierung} &\Rightarrow e \cdot k \cdot 2^\nu \cdot 2^{-\nu} = u \\
 &(e \cdot (k \cdot 2^\nu)) \cdot 2^{-\nu} = u \\
 \text{in integer mit } \nu = 1 &\Rightarrow \\
 (10 \cdot (0.5 \cdot 2)) &\gg 1 = (10 \cdot 1) \gg 1 \\
 &= (1010)_2 \gg 1 = (101)_2 = 5
 \end{aligned}$$

Als Skalierungsfaktoren verwendet man meist Zweierpotenzen, da die Intergerdivision durch die Skalierungskonstante dann durch Schiebeoperationen zeitgünstig ausgeführt werden kann (siehe Beispiel oben: Multiplikation mit 2^{-1} entspricht Division durch 2 und wird durch arithmetisches Rechtsschieben um ein Bit implementiert). Es ist aber nicht notwendig sich auf Zweierpotenzen zu beschränken.

Abbildung 7 zeigt das Blockschaltbild des Positionsreglers mit unterlagertem Drehzahlregler und den entsprechenden Skalierungsfaktoren. Im Blockschaltbild a) sind die Skalierungsfaktoren als Identitäten eingebracht. Im darunterliegenden Blockschaltbild b) sind die Skalierungsfaktoren jeweils einem Reglerparameter zugeschlagen und die Integration (bzw. die Summenbildung im Zeitdiskreten) mit dem Block n_i^{-1} vertauscht. Im Blockdiagramm c) wird eine Umformung durchgeführt, die den Skalierungsfaktor n_p^{-1} aufteilt in das Produkt $n_{p1}^{-1} \cdot n_{p2}^{-1}$ und den zweiten Faktor mit dem Distributiv-Gesetz über die Summationsstelle propagiert. Wie oben angedeutet muss auf Genauigkeit und Zahlenbereichsüberläufe geachtet werden. Dies wollen wir im nächsten Abschnitt am Beispiel des Positionsreglers betrachten.

4.2.2 Rundungsprobleme, Überläufe und Genauigkeit beim Positionsregler

Rundungsprobleme Beim gewählten Wert von k_{Rpos} und dem gewählten Skalierungsfaktor n_p muss untersucht werden, welcher Zahlenwert sich in der Integer-Realisierung von $k_{Rpos} \cdot n_p$ ergibt, auf welchen eigentlichen Regelparameter $\hat{k}_{Rpos} = \lfloor k_{Rpos} \cdot n_p \rfloor \cdot n_p^{-1}$ dieser Wert führt und ob diese Genauigkeit ausreichend ist ($\lfloor x \rfloor$ ist die größte ganze Zahl kleiner oder gleich x).

Mit den vorliegenden Zahlenwerten $k_{Rpos} = 0.3$ und $n_p = 2^7$ (siehe Abbildung 7) ergibt sich:

$$\lfloor k_{Rpos} \cdot n_p \rfloor = \lfloor 0.3 \cdot 2^7 \rfloor = \lfloor 38.4 \rfloor = 38 \quad \Rightarrow \quad \hat{k}_{Rpos} = 38 \cdot 2^{-7} = 0.2969$$

Es ergibt sich ein absoluter Fehler von $0.3 - 0.2969 = 3.1 \cdot 10^{-3}$. Wir haben also eine Genauigkeit von 2 Stellen, was hier als ausreichend betrachtet werden soll.

Überläufe Mit welchem Datentypen soll die Stellgröße des Positionsreglers (zwei Signale in Abbildung 7d)) implementiert werden?

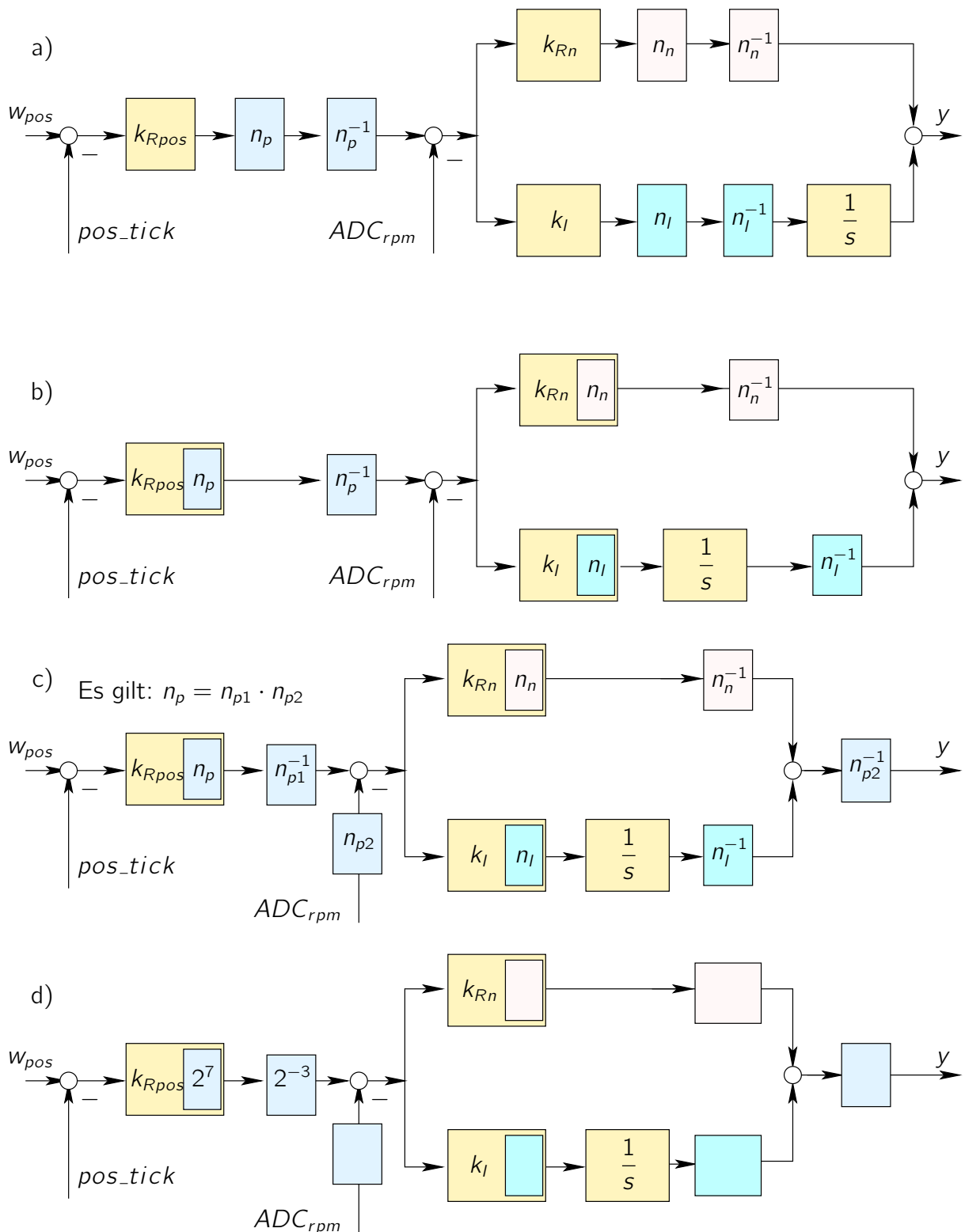


Abbildung 7: Blockschaltbildumformungen des verwendeten Positionsreglers mit unterlagertem Drehzahlregler und Skalierungsblöcken (siehe Laboraufgabe 5)

Wir versuchen es mit einem signed integer mit 16 Bit. Somit muss der Betrag des Stellgrößenwerts kleiner gleich $2^{15} - 1$ sein. Das führt auf einen maximalen Betrag des Regelfehlers von

$$\frac{2^{15} - 1}{\lfloor k_{Rpos} \cdot n_p \rfloor} = \frac{2^{15} - 1}{38} = 862.2.$$

Dies würde bei 400 Ticks pro Umdrehung einem Regelfehler von nur etwas mehr als zwei Umdrehungen entsprechen.

Wir versuchen es alternativ mit einem signed integer mit 32 Bit. Jetzt muss der Betrag des Stellgrößenwerts kleiner gleich $2^{31} - 1$ sein. Das führt auf einen maximalen Betrag des Regelfehlers von

$$\frac{2^{31} - 1}{\lfloor k_{Rpos} \cdot n_p \rfloor} > 56 \cdot 10^6.$$

Das entspricht einem Fehler von mehr als 140 Tausend Umdrehungen. Da der Sollwert der Winkelposition in Ticks der Welle für den Regler eine Eingangsgröße ist, begrenzen wir den Betrag des Positionsregelfehlers in Ticks auf $56 \cdot 10^6$. Damit kann kein Überlauf im Positionsregler entstehen.

Genauigkeit bei kleinen Positionsfehlern Der „Trick“ mit dem Aufspalten des Skalierungsfaktors n_p in $n_p = n_{p1} \cdot n_{p2}$ in Abbildung 7c) verhindert, dass, wenn wie in unserem Fall die Reglerverstärkung $\hat{k}_{Rpos} < 1$ ist und die Regeldifferenz des Positionsreglers nur ein Tick beträgt, die Stellgröße des Positionsreglers nach Integer-Umwandlung verschwindet und der Regler nicht mehr stationär genau sein kann. Falls der Positionsfehler genau ein Tick ist, ergibt sich hier eine Stellgröße des Positionsreglers von

$$\lfloor 1 \cdot \lfloor k_{Rpos} \cdot n_p \rfloor \cdot n_{p1}^{-1} \rfloor = \lfloor 38 \cdot 2^{-3} \rfloor = 4$$

Vor allem bei I-Gliedern, bei denen normalerweise die Verstärkungen k_I kleine Werte besitzen, muss darauf geachtet werden, dass es nicht zu Unterläufen kommen kann und immer 0 addiert wird, weil dadurch die Funktion des I-Gliedes nicht mehr erfüllt wird.

Bemerkung: In den Blockschaltbildern nach Abbildung 7 ist nur der Positionsregler mit unterlagertem Drehzahlregler betrachtet. Falls der Drehzahlregler alleine verwendet werden soll, entfallen die Skalierungen mit n_{p2} .

Laboraufgabe 5: Analyse des Drehzahlreglers (Datei *Lab2_Modell/zPlawu_n.int.c*)

Analysieren Sie den Drehzahlregler in der Datei *Lab2_Modell/zPlawu_n.int.c* und finden Sie die Werte der Skalierungsfaktoren n_n, n_I heraus. Tragen Sie diese in das Blockschaltbild d) der Abbildung 7 ein und ergänzen Sie die Skalierungen in den noch leeren P-Blöcken des Blockschaltbildes.

Laboraufgabe 6: Analyse der Skalierungsfaktoren des Drehzahlreglers

Ermitteln Sie die absoluten Fehler in den Parametern $k_{Rn} = 1.2$ und $k_I = k_{Rn} \frac{1.536\text{msec}}{51.1\text{msec}} = 0.036071$ durch die Integer-Realisierungen

Fehler bei k_{Rn} : 1.2–

Fehler bei k_I : 0.036071–

Nehmen Sie an, die internen Signale des Drehzahlreglers werden mit dem Datentyp int32 (long) implementiert. Welchen maximalen Betrag darf der Regelfehler haben, damit es zu keinem Überlauf kommt?

Für den P-Anteil:

Für den I-Anteil:

Ergebnis P- und I-Anteil zusammen:

In der Implementierung des Reglers wird der Regelfehler auf einen deutlich kleineren Wert begrenzt.

Laboraufgabe 7: Integer-Realisierung des Positionsreglers in Simulink

Implementieren Sie den Positionsregler mit den in Abbildung 7d) gegebenen Skalierungsfaktoren in der Datei *Modell/zP_pos_int.c*.

Im Modell *Regelkreis_ZD_C_int.slx* sind die restlichen Skalierungen bereits richtig eingetragen.

Simulieren Sie zunächst die Sprungantwort des Regelkreises mit Drehzahlregelung und dann mit Positionsregelung und vergleichen Sie die Ergebnisse mit den Simulationsergebnissen mit dem reinen Simulinkmodell *Regelkreis_ZK.slx*. Falls größere Abweichungen auftreten, überprüfen Sie zunächst, ob Sie bei beiden Modellen dieselben Reglerparameter eingestellt haben und dann Ihre Skalierungsfaktoren.

Laboraufgabe 8: C-Programm in Integer-Arithmetik auf Dragon12

Tragen Sie die Skalierungsfaktoren in den C-Code für den Mikrocontroller unter *Lab2_uC_integer/Sources/main.c* ein. Sie müssen in dieser Datei nur an den mit *TODO* markierten Stellen Änderungen durchführen.

Probieren Sie den Regler auf dem Board aus. Durch die Integer-Realisierung sollte das Regelverhalten praktisch unverändert bleiben.

Vervollständigen Sie die Tabelle 9 mit den Rechenzeiten der beiden Regler in Integer-Realisierung.

Laboraufgabe 9: Laufzeiten der verschiedenen Dragon12-Programme

Tragen Sie die gemessenen Werte der Laufzeiten der C-Programme für die beiden Betriebs-

arten der Regelung in die folgende Tabelle ein:

Reglerart	C-Code mit float-Variablen	Autocode mit float-Variablen	C-Code mit integer-Variablen
Drehzahlregelung			
Positionsregelung			

Hinweis: Die gemessenen Werte enthalten die reine Rechenzeit der Regler ohne die ADC-Wandungen (Medianfilter mit je 3 Strom- und 3 Spannungsmessungen), den Stimulusgenerator für die sprungförmigen Sollwerte usw., die ebenfalls innerhalb der RTI-Interrupt-Service-Routine implementiert wurden. Dafür sowie für die serielle Datenübertragung des SoftScopes werden weitere ca. $800\mu\text{sec}$ benötigt.

5 Anhang: Analyse von Messdaten

5.1 Software-Oszilloskop

Interne Werte des Mikrocontrollerprogramms, z.B. die mit dem ADC gemessene Motordrehzahl, die Solldrehzahl oder die Istposition, können über die serielle Schnittstelle im Echtzeitbetrieb an den PC übertragen und in Matlab angezeigt werden. Dazu wird **im Mikrocontrollerprogramm** am Ende der Interrupt-Service-Routine `myADC1Callback()`, die im RTI-Abtastraster alle $T = 1536\mu\text{sec}$ ausgeführt wird, die Funktion

```
SoftScope((int16) set_rpm, (int16) measured_rpm);
```

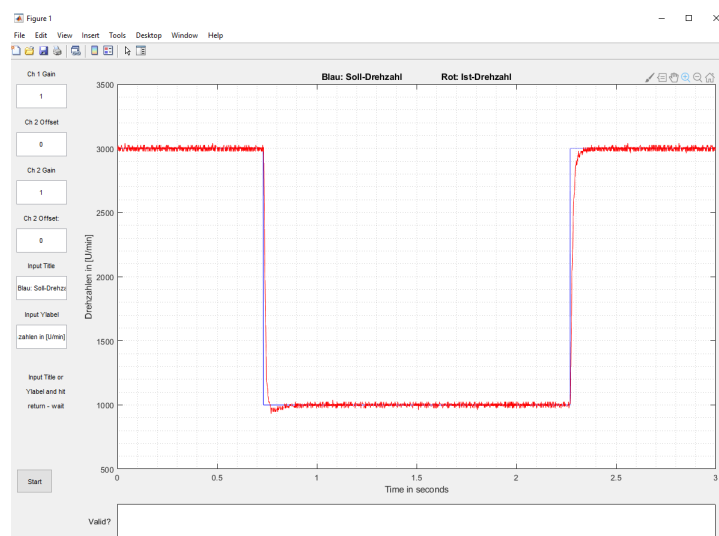
aufgerufen. Beachten Sie, dass die übertragenen Werte in 16bit Integerzahlen konvertiert werden. Für eine Übertragung von Gleitkommazahlen oder von mehr als zwei Werten innerhalb der Abtastzeit ist die serielle Schnittstelle (115kbit/sec) zu langsam.

Das Dragon12-Mikrocontrollerboard muss zusätzlich zur Debugger-Schnittstelle über die zweite serielle Schnittstelle mit dem PC verbunden werden. **In der Matlab-Konsole** muss die folgende Funktion für das Software-Oszilloskop ausgeführt werden:

```
SoftScope(comIF, ...)
```

Als erster Parameter muss die Nummer der seriellen Schnittstelle, z.B. `comIF=2`, übergeben werden. Falls Sie die Nummer nicht kennen, können Sie `comIF=0` setzen, dann werden alle vorhandenen Schnittstellen aufgelistet. Die dargestellten Zeitsignale sind die vom Mikrocontroller-Programm gesendeten 16bit-Integerwerte. Über die Bedienoberfläche des Programmfensters bzw. weitere Aufrufparameter (Gain und Offset) der Funktion (Hilfe mit `help SoftScope`) können Sie die Werte umskalieren, z.B. in U/min .

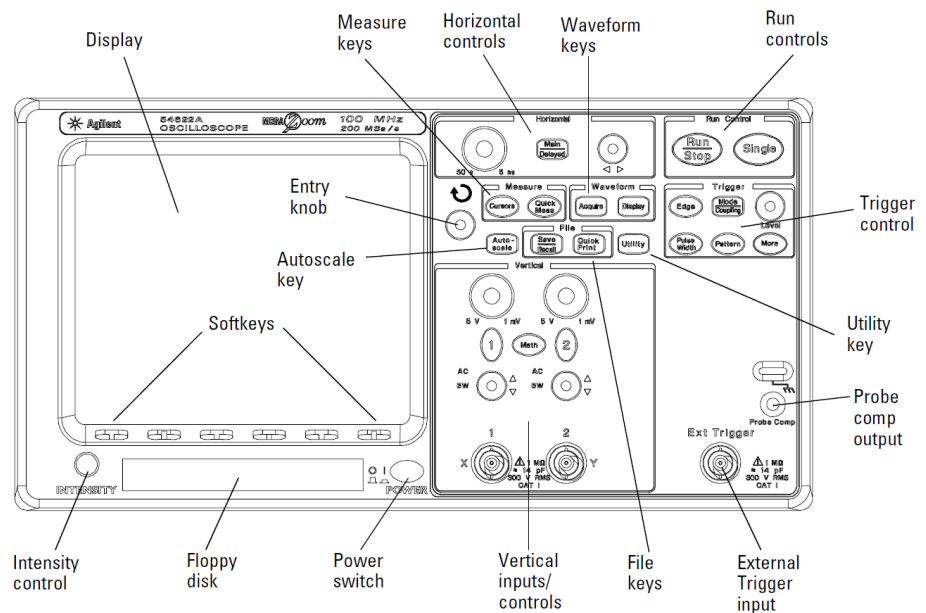
Das Bildschirmfenster zeigt einen Zeitbereich von einigen Sekunden an und wird laufend aufgefrischt. Um Details näher analysieren zu können, stoppen Sie das Oszilloskop über den Start/Stop-Button im Programmfenster und zoomen Sie im Bild mit den üblichen Matlab-Werkzeugen.



Achtung: Falls das Software-Oszilloskop hängen bleibt, schließen Sie das Fenster und starten Sie es neu. Eventuell müssen Sie anschließend `clear all` ausführen oder sogar Matlab neu starten.

5.2 Bedienung des Digital-Speicher-Oszilloskops

Die Bedienung erfolgt über Tasten (**Hardkeys**) und Drehknöpfe (**Knobs**) auf der Vorderseite des Gerätes sowie Menütasten (**Softkeys**) unterhalb des Displays. Wenn eine Menütaste mehrere Optionen hat, wählt man diese über den Drehknopf **Entry knob**.



Grundeinstellungen:

- **Tastköpfe anschließen** und mit Messsignalen verbinden. Zugehörige Kanäle über Hardkeys 1 und 2 einschalten.
- Hardkey **Autoscale** drücken: Stellt Kanalverstärkung und Trigger automatisch ein, funktioniert aber nur bei „schnellen“ Signalen, z.B. den TTL-Signalen des Positionsgebers. Für Messung von Sollwertsprüngen der Drehzahl mit dem Tachogebersignal nicht geeignet.
- Erneutes Drücken von Autoscale schaltet auf die vorige manuelle Einstellung zurück.

Kanalverstärkung und Zeitablenkung:

- Die **Verstärkung** kann für jeden Kanal an den großen Drehknöpfen im Bedienfeld Vertical eingestellt werden. Mit den darunterliegenden kleinen Drehknöpfen kann für jeden Kanal der **Offset**, d.h. die vertikale Lage, verstellt werden.
- Die **Zeitablenkung** wird mit dem großen Drehknopf im Bedienfeld Horizontal eingestellt.
- Die eingestellten Werte (V/Div bzw. s/Div) werden im Display in der oberen Zeile angezeigt.

Trigger:

- Hardkey **Edge** wählt man das Menü im Display aus. Mit Softkey **Rising** ↑ bzw. **Falling** ↓ kann die steigende oder fallende Signalflanke zum Triggern ausgewählt werden. Mit den Softkeys 1 bzw. 2 wählt man den **Triggerkanal** aus. Der **Triggerpegel** kann über den Drehknopf Level im Bedienfeld Trigger eingestellt werden.
- Hardkey **Mode** wählt das Menü im Display aus. Über den Softkey Mode wählt man Auto oder Normal (für langsame Signale wie Sollwertsprünge besser geeignet). Bei **Coupling** sollte DC ausgewählt werden.

6 Anhang: Bedienoberfläche des HCS12-Programms

Bedientasten:

- Taste SW2 (ganz links):
Umschalten zwischen Drehzahlregelung (R), Positionsregelung (P) und Handbetrieb (M)
- Taste SW3:
Umschalten zwischen automatischer Sollwerterzeugung (eingebauter Rechteckgenerator) oder Sollwertvorgabe über SW4/SW5.
- Taste SW4:
Sollwert vergrößern
- Taste SW5 (ganz rechts):
Sollwert verringern

Anzeige im LCD-Display obere Zeile:

- Im RPM-(Drehzahlregler)-Modus:
R: w =Solldrehzahl e =Regelfehler (beide in U/min)
- Im Positionsregler-Modus:
P: w =Sollposition e =Regelfehler
- Im Manual-(Hand)-Modus:
M: u =manuell vorgegebener PWM-Wert n =gemessene Drehzahl (in U/min)

Anzeige im LCD-Display untere Zeile:

- Bedienhinweise zu den Tasten SW4, SW5 / Fehlermeldung / Drehzahl aus TTL-Signalen

LEDs (am Port B, Ansteuerung siehe Programmcode):

- LED7 (ganz links): Blinkt ungefähr im Sekundentakt
- LED6: Blinkt mit Abtastfrequenz (= wirkt optisch wie dauernd an)
- LED5: Im Abtastraster eingeschaltet, solange Regleralgorithmus berechnet wird (= wirkt optisch wie dauernd an). Für Laufzeitmessungen der Rechenzeit mit dem Oszilloskop.
- LED4: Im Abtastraster von Beginn bis Ende der seriellen Datenübertragung (SoftScope) eingeschaltet. Für Zeitmessungen mit dem Oszilloskop.
- LED3: Fehler in der Drehrichtungserkennung (Plausibilität Tachosignal - Positionsgeber). Gelbes und graues Kabel des Positionsgebers vertauscht?
- LED1: Blinkt im Takt des Sollwertgenerators, kann zum Triggern bei Sollwertsprüngen verwendet werden.
- LED0 (ganz rechts): Ansteuerung eines Relais auf dem Motorboard, im Normalbetrieb dauernd an. Nicht ändern!

7 Anhang: Anschlüsse Motorboard - Mikrocontroller

	Motorboard bzw. Anpassschaltung	Mikrocontroller
PWM–Ansteuerung	A: PWM–Eingang (schwarzes Kabel mit Schirm)	PP0 (PWM-Ausgang) Schirm (Masse) auf GND PWM-Kanal 0
Positionsgeber	M: TTL links U_1 (graues Kabel)	PH7, PH6 und PT1 Digitaleingänge Port H.6/7 und Timer-Kanal 1
Positionsgeber	M: TTL rechts U_2 (gelbes Kabel)	PH4, PH5 und PT0 Digitaleingänge Port H.4/5 und Timer-Kanal 0
ADC–Eingang	A: Tachogenerator–Ausgang (schwarzes Kabel)	PAD08 ADC-Kanal 0 (an ATD1)
ADC–Eingang	A: Stromsensor–Ausgang (schwarzes Kabel)	PAD09 ADC-Kanal 1 (an ATD1)
Relais	A: Rotes Kabel	PB0 Digitalausgang Port B.0
	Auf Dragon12	LEDs an Port B.7...B.1 Debug-Hilfen
	Auf Dragon12	Buttons an Port H.3...H.0 Anwendereingaben
		RTI-Timer erzeugt Abtastzeit
Verbindung zum PC	Auf Dragon12	Serielle Schnittstelle SCI0 Codewarrior–Debugger

Tabelle 1: Verdrahtung zwischen Mikrocontroller und Motorboard bzw. Anpassung